UNIVERSIDAD CASTRO CARAZO 1936

Portafolio de evidencias

Programación

Britany Sofia Quirós Barahona

➢ Instrucciones

El portafolio pretende llevar un registro de los aprendizajes de cada semana, generando un documento que evidencia los distintos temas y actividades desarrolladas en clases. Cada estudiante tiene una visión particular de lo aprendido, por ende, cada portafolio representara la forma que cada uno de interpretar el conocimiento adquirido.

Este documento refleja y será parte del aprendizaje significativo, que usted como estudiante va generando a lo largo del curso, y de las habilidades que con el tiempo ira adquiriendo.

¿Qué vimos?

Esta pregunta responde al conocimiento compartido durante las lecciones, de manera teórica o práctica, por medio de los distintos medios proporcionados por el docente.

¿Qué aprendí?

Cuál fue el aprendizaje significativo, que me quedo de lo visto en clase y las distintas actividades elaboradas, tanto en las lecciones o las actividades asincrónicas.

¿Cómo lo puedo aplicar?

Una vez contestadas las preguntas anteriores, analizo y situó las habilidades y conocimientos adquiridos en mi contexto laboral o de otros indoles en la cual pueda aplicarlos y me sirvan para la mejora de mi desempeño.

© Prof. Ing. Andrés Mena A.

➢ Semana 1

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Que es un algoritmo **Como programar de manera simple por medio de un juego**	Necesito saber que quiere el cliente o que expectativas están pendientes para que este satisfecho/a/os/as. Puedo trabajar con prueba y error. Es bueno buscar la manera más fácil y rápida de llegar a un mismo resultado.	Puedo preguntarme que necesidades tienen los clientes antes de comenzar con cualquier proyecto en base a esas necesidades puedo empezar a construir una propuesta de solución.

Evidencias

Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.



© Prof. Ing. Andrés Mena A.

➢ Semana 2

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Como hacer códigos y seudocódigos **Como resolver errores basicos** **Jerga de programación (reventar, depurar etc.)**	Python tiene habilidades básicas como “int” o “input” también que tiene variables o objetos que son identificados por la computadora como nombres únicos por lo que debo identificarlos muy bien. Analizar línea por línea y seguir un pensamiento lineal y ordenador para poder identificar como resolver una necesidad o situación por medio de una entrada después un proceso y por último con una salida que es el resultado de la necesidad que trato de cubrir con el código o seudocódigo.	Puedo hacer planteamientos de las necesidades de un cliente y luego propuestas para cumplir con esas necesidades antes que comenzar directamente con una solución que casi seguramente tendrá fallos y probablemente no cubrirá la necesidad del cliente.

Evidencias

Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.

© Prof. Ing. Andrés Mena A.

2 actividades de "Reto"

DIVISOR DE CUENTA

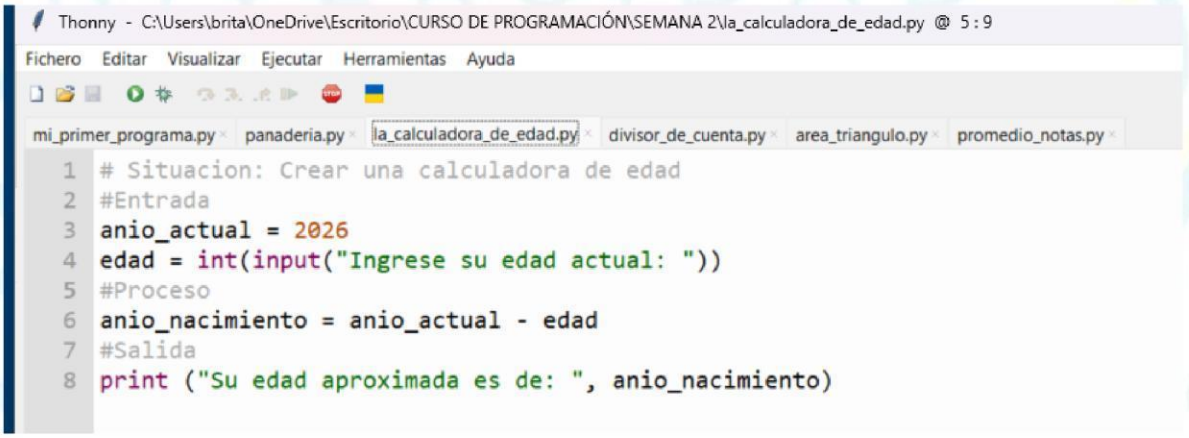
mi_primer_programa.py | panaderia.py | la_calculadora_de_edad.py | divisor_de_cuenta.py | area_triangulo.py | promedio_notas.py

```python
#Situacion: Divisor de cuenta para saber cuanto paga cada persona
#Entrada
total_personas = int(input("Ingrese el número de personas: ")
total_factura = int(input("Ingrese el total de la factura: ")
#Proceso
total_por_persona = total_personas/total_factura

#Salida
                    print("Cada uno debe pagar: ", total_por_persona)

```

CALCULADORA DE AÑO DE NACIMIENTO



ASINCRONICO

Adjunto

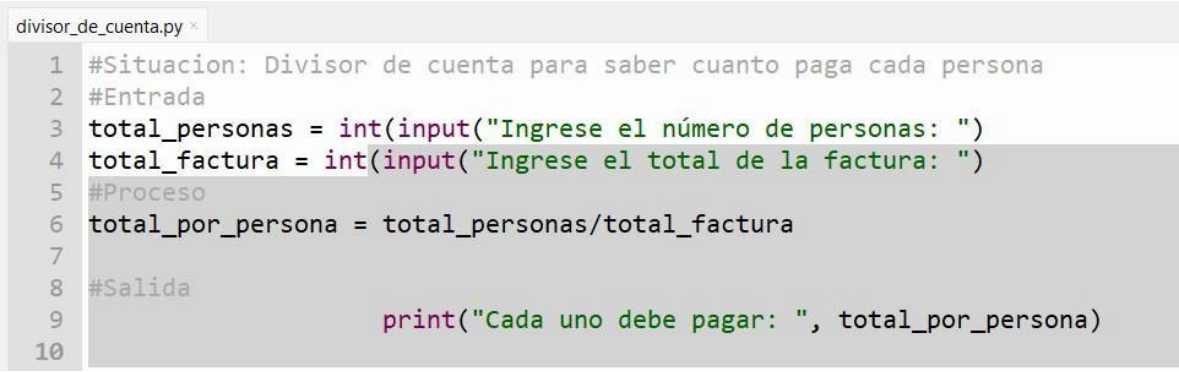
© Prof. Ing. Andrés Mena A.

Semana 3

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Operadores condicionales y Relacionales Modulo 1 Cisco Net Acad Examen 1	Importancia de Python en el mercado laboral actual y en el sector tecnológico Diferencias entre un lenguaje de máquina y un lenguaje humano, así como sobre las diferencias y los componentes de un lenguaje.	Solución de procesos repetitivos y automatización. Ahora puedo aplicar lo que aprendí en Tonny en Python como tal con programas sencillos o condicionales de if, elif, or, and. Depuración de datos y clasificación de información cuantitativa o cualitativa

Evidencias

Acá pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.



control_ingreso_mayor_de _edad.py

```python
#COntrol de ingreso mayor de edad
edad = int(input("Ingrese su edad: "))

if edad>=18:
    print("Bienvenido al concierto!!")

if edad<18:
    print("Lo sentimos, este recinto es para personas mayores de edad.")

```

reto_descuentos.py

```python
#Calculadora de descuentos
#mayor a 1000 20 de descuento
# entre 500 y 1000 10%

monto= float(input("Por favor, ingrese el monto de la compra: "))

if monto <0:
    print ("Error: El monto debe ser positivo, mayor a cero")
else:
    if monto>=1000:
        descuento = monto*0.2
    elif monto>=500:
        descuento= monto*0.1
    else:
        descuento = 0
    print("Monto: ",monto)
    print("Descuento: ", descuento)
    print("Total a pagar: ", monto-descuento)


```

reto_visado.py

```python

#programar el Sistema de Aprobación de Visas
#Entrada
print ("|||BIENVENIDO A NUESTRO FORMULARIO DE APROBACIÓN DE VISAS|||")
print ("")
print("Por favor, ingrese los datos personales que se le solicitan a continuación:")
print ("")
print ("En caso de no tener respuesta por favor click en tecla enter.")
print ("")
print ("Por favor, recuerde siempre ingresar sus respuestas con minusculas y mayusculas según corresponda.")
print ("")
monto = int(input("Por favor, ingrese el monto disponible en su cuenta bancaria ($)"))
antecedentes = input("¿Tiene antecedentes penales? (Si/No): ")
profesion = input("Por favor, ingrese su profesión: ")
print ("")


#CASO 1 (Visa Dorada):
if (monto >=50000 and antecedentes == "No" or profesion == "Diplomatico"):
    print("APROBADO: VISA DORADA.")

#Caso 2 CASO 2 (Visa Estándar):
elif (monto >= 20000 and antecedentes == "No" or profesion == ""):
    print("APROBADO: VISA ESTANDAR.")

```

Consola

```python
# CASO 3 (Revisión Especial):
elif(monto < 20000 and profesion == "Estudiante" or profesion == "Jubilado"):
    print("PENDIENTE:Requiere entrevista.")

#CASO 4 (Rechazo):
else:
    print ("Su VISA ha sido denegada.")

print ("")
print ("|||GRACIAS POR USAR NUESTRO SISTEMA.|||")
```

Semana 4

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Habilidades: **strip()** **Quita espacios.** **lower()** **Convierte a minúsculas.** **and tiene que cumplir las dos .** **or** **Cualquiera de las dos.**	Se pueden evitar los errores de entrada de información de los usuarios. Se pueden crear condicionales que admitan mas de una condición.	Sirve para depurar datos
Como usar github.	Como crear repositorios, editaros y subir información a la nube	Asi puedo administrar datos, proyectos y cambios en archivos euna manera mucho mas ordenada.
Modulo 2 CISCO	En el Módulo 2 se estudiaron las variables, tipos de datos (int, float, str), entrada y salida de datos con input() y print(), conversión de tipos (int(), float()), operadores aritméticos (+, -, *, /, //, %, **), prioridad de operadores y resolución de expresiones matemáticas utilizando Python.	Guardar información en variables. Pedir datos al usuario con input(). Mostrar resultados con print(). Hacer cálculos matemáticos. Resolver problemas reales como presupuestos, promedios, tiempos, descuentos y salarios.

Evidencias

Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.

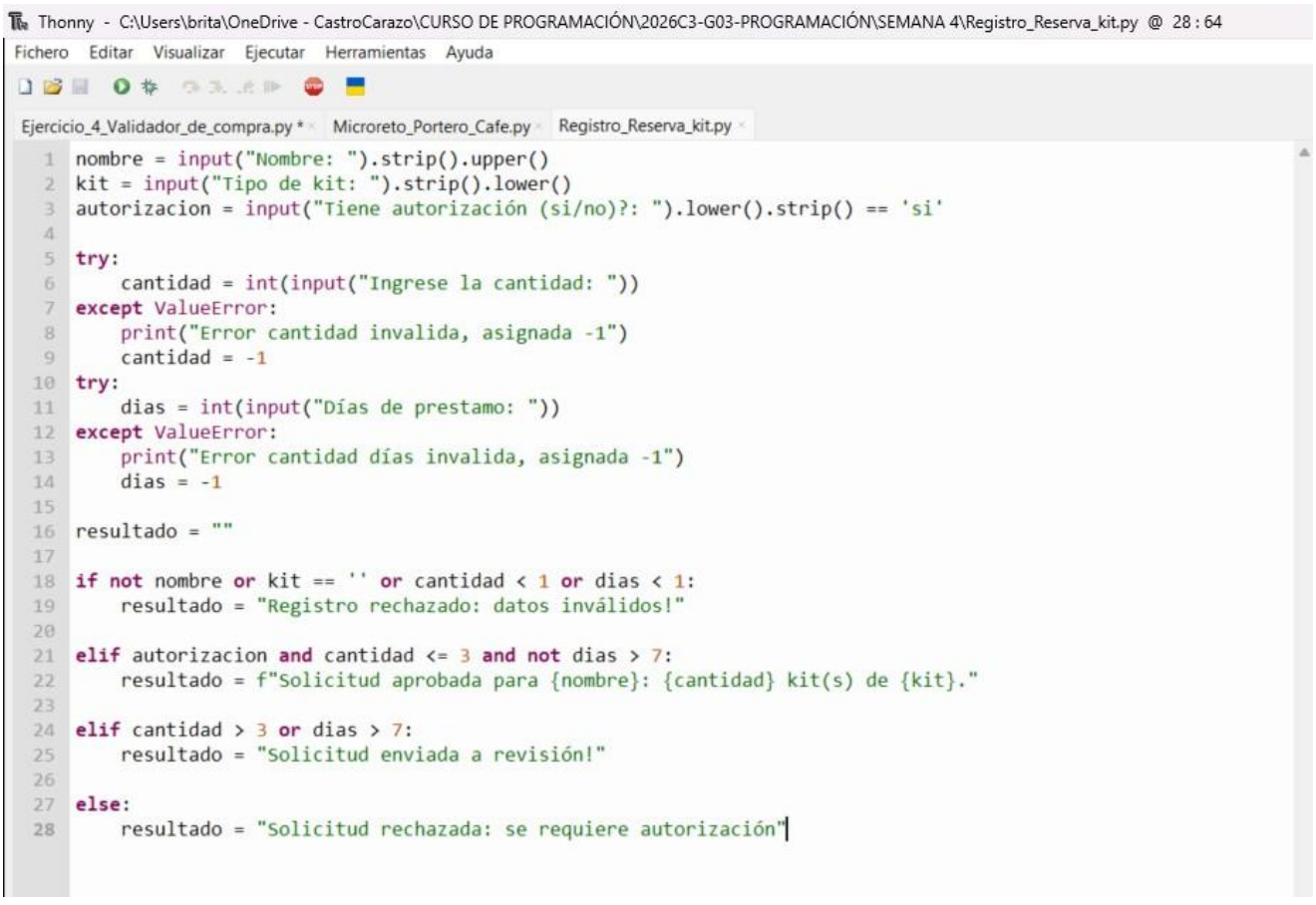
Ejercicio_4_Validador_de_compra.py * | Microreto_Portero_Cafe.py

```python
"""Microreto: el portero del café."""

energia = int(input("Cuanta enería tienes de 0-100: "))
trae_cafe = input("¿Traes café? (si/no): ").lower().strip() == "si"

mensaje = "Completa las reglas del portero."

if energia < 30 and not (trae_cafe):
    print("No puede pasar!")
elif energia >= 30 or trae_cafe :
    print("Bienvenido, puede pasar!")
else:
    mensaje= "Portero confundido!, revisar las respuestas."
```



Tarea Semana 4: Segundo Análisis de Casos

Código limpio, datos de texto y validación

Nombre: Britany Sofia Quiros Barahona Curso: 2026C3-G03-PROGRAMACIÓN

Fecha: 24 Octubre 2026

Este trabajo utiliza conceptos básicos de Python: input(), int(), float(), try-except, condicionales if/elif/else, operadores, strip(), lower() y f-strings.

Problema #1: Calculadora de propina inteligente

Problema

Crear un programa que calcule cuánto se debe dejar de propina y cuál será el total final de una cuenta.

Entrada

El programa solicita el monto de la cuenta y el porcentaje de propina.

Proceso

Convierte los datos a números decimales con float. Verifica que la cuenta y el porcentaje no sean negativos y que el porcentaje no supere 100. Calcula la propina con cuenta × porcentaje / 100 y suma la propina a la cuenta.

Salida esperada

Muestra la propina y el total final con dos decimales. Si se escribe un dato no numérico o fuera de los límites, muestra un mensaje de error.

Código

```
try:
    cuenta = float(input("Ingrese el total de la cuenta: "))
    porcentaje = float(input("Ingrese el porcentaje de propina: "))

    if cuenta < 0 or porcentaje < 0 or porcentaje > 100:
        print("Error: revise la cuenta y el porcentaje de propina.")
    else:
        propina = cuenta * porcentaje / 100
        total_final = cuenta + propina
        print(f"Propina: ■{propina:.2f}")
        print(f"Total final: ■{total_final:.2f}")

except ValueError:
    print("Error: debe ingresar números válidos.")
```

Ejemplo esperado: Ejemplo: cuenta ■10 000 y propina 10 % → propina ■1 000,00 y total ■11 000,00.

Problema #2: Detector de número par/impar

Problema

Crear un programa que solicite un número entero e indique si es par o impar.

Entrada

El usuario escribe un número entero.

Proceso

Convierte la entrada con int. Si el residuo de dividir el número entre 2 es cero (numero % 2 == 0), es par; de lo contrario, es impar. try-except permite manejar entradas que no sean enteros.

Salida esperada

Muestra si el número es par o impar, o un mensaje de error si la entrada no es un entero válido.

Código

```
try:
    numero = int(input("Ingrese un número entero: "))

    if numero % 2 == 0:
        print(f"El número {numero} es par.")
    else:
        print(f"El número {numero} es impar.")

except ValueError:
    print("Error: debe ingresar un número entero válido.")
```

Ejemplo esperado: Ejemplo: entrada 7 → "El número 7 es impar."

Problema #3: Clasificador de edad

Problema

Crear un programa que clasifique la edad como menor de edad, adulto o adulto mayor.

Entrada

El usuario ingresa su edad en años.

Proceso

Convierte la entrada a entero y verifica que no sea negativa. Si es menor que 18, es menor de edad; si es menor que 65, es adulto; en caso contrario, es adulto mayor.

Salida esperada

Muestra la categoría correspondiente. Si la edad es negativa o no es un entero válido, muestra un mensaje de error.

Código

```
try:
    edad = int(input("Ingrese su edad en años: "))

    if edad < 0:
        print("Error: la edad no puede ser negativa.")
    elif edad < 18:
        print("Clasificación: menor de edad.")
    elif edad < 65:
        print("Clasificación: adulto.")
    else:
        print("Clasificación: adulto mayor.")

except ValueError:
    print("Error: debe ingresar una edad usando números enteros.")
```

Ejemplo esperado: Ejemplo: entrada 21 → "Clasificación: adulto."

Problema #4: Validador de compra con descuento

Problema

Solicitar el nombre del cliente, el monto de la compra y si es cliente frecuente. Aplicar un descuento según las reglas definidas.

Entrada

El programa solicita nombre, monto de compra y respuesta si/no. Se aceptan "sí", "si" y "no", sin importar espacios al inicio/final o mayúsculas.

Proceso

Limpia el nombre con strip() y convierte la respuesta a minúsculas con lower(). Si el nombre está vacío o el monto es negativo, muestra un error. Si es cliente frecuente y compra ■50 000 o más, recibe 10 %. Si solo cumple una condición, recibe 5 %. Si no cumple ninguna, recibe 0 %. Después calcula descuento y total.

Salida esperada

Muestra el nombre, porcentaje y monto del descuento, y total a pagar. Los datos inválidos generan un mensaje claro.

Código

```
try:
    nombre = input("Ingrese el nombre del cliente: ").strip()
    monto = float(input("Ingrese el monto de la compra: ■"))
    respuesta = input("¿Es cliente frecuente? (si/no): ").strip().lower()

    if nombre == "":
        print("Error: el nombre no puede estar vacío.")
    elif monto < 0:
        print("Error: el monto no puede ser negativo.")
    elif respuesta != "si" and respuesta != "sí" and respuesta != "no":
        print("Error: responda si o no.")
    else:
        if respuesta == "si" or respuesta == "sí":
            frecuente = True
        else:
            frecuente = False

        if frecuente and monto >= 50000:
            descuento_porcentaje = 10
        elif frecuente or monto >= 50000:
            descuento_porcentaje = 5
        else:
            descuento_porcentaje = 0

        descuento = monto * descuento_porcentaje / 100
        total_pagar = monto - descuento

        print(f"Cliente: {nombre.title()}")
        print(f"Descuento aplicado: {descuento_porcentaje}% (■{descuento:.2f})")
        print(f"Total a pagar: ■{total_pagar:.2f}")

except ValueError:
    print("Error: el monto debe ser un número válido.")
```

Ejemplo esperado: Pruebas: ■60 000 y sí → 10 %; ■60 000 y no → 5 %; ■30 000 y sí → 5 %; ■30 000 y no → 0 %.

Pruebas manuales

Caso	Monto	Frecuente	Resultado esperado
1	■60 000	Sí	10 %; paga ■54 000
2	■60 000	No	5 %; paga ■57 000
3	■30 000	Sí	5 %; paga ■28 500
4	■30 000	No	0 %; paga ■30 000
5	-■100	Sí	Error por monto negativo
6	Texto	Sí	Error por dato no numérico

Conclusión

La validación más importante fue comprobar que los datos numéricos fueran válidos y que no fueran negativos cuando correspondía. Durante las pruebas, un posible error es escribir texto en lugar de un número o una respuesta distinta de "sí" o "no". La limpieza del código con nombres claros, strip() y lower() ayuda a evitar errores por espacios o mayúsculas y hace que el programa sea más fácil de leer y mantener.

Ejercicio_4_Validador_de_compra.py * | Microreto_Portero_Cafe.py | Registro_Reserva_kit.py | Diagnostico_Semanal_Ventas.py | Ejercicio_4_Validador_de_compra.py | A

```python
# Ejercicio 4: Validador de compra con descuento

try:
    nombre = input("Ingrese el nombre del cliente: ").strip()
    monto = float(input("Ingrese el monto de la compra: ₡"))
    respuesta = input("¿Es cliente frecuente? (si/no): ").strip().lower()

    if nombre == "":
        print("Error: el nombre no puede estar vacío.")
    elif monto < 0:
        print("Error: el monto no puede ser negativo.")
    elif respuesta != "si" and respuesta != "sí" and respuesta != "no":
        print("Error: responda si o no.")
    else:
        if respuesta == "si" or respuesta == "sí":
            frecuente = True
        else:
            frecuente = False

        if frecuente and monto >= 50000:
            descuento_porcentaje = 10
        elif frecuente or monto >= 50000:
            descuento_porcentaje = 5
        else:
            descuento_porcentaje = 0

        descuento = monto * descuento_porcentaje / 100
        total_pagar = monto - descuento

        print(f"Cliente: {nombre.title()}")
        print(f"Descuento aplicado: {descuento_porcentaje}% (₡{descuento:.2f})")
        print(f"Total a pagar: ₡{total_pagar:.2f}")

except ValueError:
    print("Error: el monto debe ser un número válido.")
```

MODULO 2 COMPLETADO:



https://github.com/Sofqb13/2026C3-G03-PROGRAMACI-N

Semana 6

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Vimos como usar los ciclos for y while para repetir cosas y no tener que hacer todo a mano. Usamos el for para pasar minutos a horas o filtrar datos, y el while para ir actualizando listas, pedir datos hasta que uno escriba "fin" y borrar cosas que no sirvieran. También hicimos retos para controlar el aforo de un lugar viendo que no se pasara del límite.	Aprendí cuando usar cada ciclo; por ejemplo, el for sirve más cuando uno sabe qué recorrer y el while cuando no se sabe cuántas veces se va a repetir la cosa (como cuando ponemos "fin" para salir). También le agarré el toque a usar bien las listas con append() y a mezclar todo esto con los if y el try-except para que el programa no se pegue si alguien mete un dato malo.	Lo puedo usar en el trabajo para automatizar tareas repetitivas con datos y limpiar archivos sin estar haciéndolo uno por uno. También para crear validaciones cuando la gente mete información y asegurarme de que cumpla con lo que se pide antes de guardarla.

Evidencias

Aca pueden aportar imágenes, tablas o cualquier material de la evidencia de su aprendizaje.

O de la aplicación de alguno de los conceptos vistos en su vida profesional o personal.

control_aforo.py | while_03_eliminar_incompletas.py * | for_01_transformar_datos.py

```python
"""Micro-reto 1: transformar datos con for.

Contexto: datos ficticios de minutos de estudio registrados en un curso
universitario de UMCA.

Prediga el resultado, complete el ciclo y compruebe su respuesta.
"""

minutos_estudio = [60, 120, 90]
horas_estudio = []

for minutos in minutos_estudio:
    horas = minutos / 60
    horas_estudio.append(horas)
    # TODO: convierta los minutos a horas y agregue el resultado a la lista.
    print(minutos, " - >", horas)

print("Horas de estudio:", horas_estudio)

# Resultado esperado: [1.0, 2.0, 1.5]

```

control_aforo.py | while_03_eliminar_incompletas.py * | for_01_transformar_datos.py | for_02_seleccionar_datos.py

```python
"""Micro-reto 2: seleccionar datos con for e if.

Contexto: datos ficticios de horas de estudio semanal reportadas en un curso
universitario de UMCA. Se desea identificar los registros de 3 horas o más.

Prediga cuántos registros cumplen la condición antes de completar el ciclo.
"""
#                    0  1  2  3  4
horas_reportadas = [2, 4, 3, 1, 5]
horas_meta = []

for horas in horas_reportadas:
    if horas >= 3:
        # TODO: agregue a horas_meta el dato que cumple la condición.
        horas_meta.append(horas)

print("Registros que alcanzan la meta:", horas_meta)
print("Cantidad de registros:", len(horas_meta))

# Resultado esperado:
# Registros que alcanzan la meta: [4, 3, 5]
# Cantidad de registros: 3

```

control_aforo.py | while_03_eliminar_incompletas.py * | for_01_transformar_datos.py | for_02_seleccionar_datos.py | while_01_actual

```python
"""Micro-reto 3: actualizar una lista por índice con while.

Contexto: puntajes ficticios de una actividad universitaria expresados como
porcentajes. Para un análisis, se transformarán a valores entre 0 y 1.

Prediga la lista final. Complete la actualización y siga el valor del índice.
"""

puntajes = [72, 85, 90]
indice = 0

while indice < len(puntajes):
    # TODO: actualice el puntaje de esta posición dividiéndolo entre 100.
    pass

    indice += 1

print("Puntajes normalizados:", puntajes)

# Resultado esperado: [0.72, 0.85, 0.9]

```

control_aforo.py | while_03_eliminar_incompletas.py * | for_01_transformar_datos.py | for_02_seleccionar_datos.py | while_01_actualizar_lista.py | while_02_reunir_datos.py | Asistente

```python
"""Micro-reto 4: reunir datos hasta recibir una palabra de cierre.

Contexto: se registran resultados ficticios de una actividad universitaria.
La cantidad de registros no se conoce al iniciar.

Ingrese enteros positivos y escriba "fin" para terminar. No ingrese texto
distinto de "fin" ni números negativos en esta práctica.
"""

resultados = []
entrada = input("Resultado (o fin para terminar): ").strip().lower()

while entrada != "fin":
    resultado = int(entrada)
    # TODO: agregue el resultado a la lista.
    resultados.append(resultado)
    pass
    entrada = input("Resultado (o fin para terminar): ").strip().lower()

print("Resultados registrados:", resultados)
print("Cantidad de resultados:", len(resultados))

if resultados:
    promedio = sum(resultados) / len(resultados)
    print(f"Promedio: {promedio:.2f}")
else:
    print("No se ingresaron resultados.")
# Con las entradas 80, 65, 90 y fin, se espera:
# Resultados registrados: [80, 65, 90]
# Cantidad de resultados: 3
# Promedio: 78.33

```

control_aforo.| while_03_eliminar_incompleta | for_01_transformar_datc | for_02_seleccionar_datc | while_01_actualizar_list | while_02_reunir_dato | while_03_eliminar_incomplet | Asistente

```python
"""Micro-reto 5: eliminar registros incompletos con while y remove().

Contexto: respuestas ficticias de una encuesta universitaria sobre recursos
de estudio. Para este resumen se excluirán las respuestas "Sin respuesta".

Prediga la lista final y cuántas respuestas incompletas se eliminarán.
"""

respuestas = ["Bueno", "Sin respuesta", "Excelente", "Sin respuesta", "Regular"]
cantidad_incompletas = respuestas.count("Sin respuesta")
respuestas.remove("Sin respuesta")

while "Sin respuesta" in respuestas:
    # TODO: elimine una respuesta "Sin respuesta" de la lista.
    respuestas.remove("Sin respuesta")

print("Respuestas para analizar:", respuestas)
print("Respuestas incompletas eliminadas:", cantidad_incompletas)

# Resultado esperado:
# Respuestas para analizar: ['Bueno', 'Excelente', 'Regular']
# Respuestas incompletas eliminadas: 2

```

PE1: Módulo 1. Introducción a Python y a la Programación Informática

100%

PE1: Módulo 2. Tipos de datos, Variables, Operaciones Básicas de Entrada y Salida, Operadores Básicos

100%

PE1: Módulo 3. Valores Booleanos, Ejecución Condicional, Bucles, Listas y su procesamiento, Operaciones Lógicas y de Bit a Bit

100%

control_aforo.p | while_03_eliminar_incomplet | for_01_transformar_datc | for_02_seleccionar_datc | while_01_actualizar_li

```python
# Reto integrador: control de ingreso al Anfiteatro del CENAC
# Estudiante: Britany Sofía Quirós Barahona

capacidad_maxima = 700
ocupacion_actual = 0

grupos_aceptados = []
grupos_rechazados = []

print("=== CONTROL DE AFORO CENAC ===")
entrada = input("Ingrese la cantidad de personas del grupo (o fin): ")

while entrada != "fin":
    try:
        grupo = int(entrada)

        if grupo <= 0:
            print("Error: El número debe ser mayor a cero.")
        else:
            # Revisar si cabe en el aforo
            if ocupacion_actual + grupo <= capacidad_maxima:
                ocupacion_actual = ocupacion_actual + grupo
                grupos_aceptados.append(grupo)
                print("Grupo aceptado: ingresan", grupo, "personas.")
            else:
                grupos_rechazados.append(grupo)
                print("Grupo rechazado: no hay espacio para", grupo, "personas.")

            # Mostrar ocupación y espacios
            espacios_disponibles = capacidad_maxima - ocupacion_actual
            print("Ocupación actual:", ocupacion_actual)
            print("Espacios disponibles:", espacios_disponibles)
            print("")

    except ValueError:
        print("Error: Entrada inválida. Ingrese un número o escriba fin.")

    entrada = input("Ingrese la cantidad de personas del grupo (o fin): ")

# Calcular total admitido usando for
total_admitido = 0
for g in grupos_aceptados:
    total_admitido = total_admitido + g

espacios_finales = capacidad_maxima - total_admitido

# Reporte final
print("")
print("REPORTE FINAL")
print("Grupos aceptados:", len(grupos_aceptados))
print("Grupos rechazados:", len(grupos_rechazados))
print("Personas admitidas:", total_admitido)
print("Capacidad máxima:", capacidad_maxima)
print("Espacios disponibles:", espacios_finales)
```

Consola

```
=== CONTROL DE AFORO CENAC ===
Ingrese la cantidad de personas del grupo (o fin): 650
Grupo aceptado: ingresan 650 personas.
Ocupación actual: 650
Espacios disponibles: 50

Ingrese la cantidad de personas del grupo (o fin): 60
Grupo rechazado: no hay espacio para 60 personas.
Ocupación actual: 650
Espacios disponibles: 50

Ingrese la cantidad de personas del grupo (o fin): fin

REPORTE FINAL
Grupos aceptados: 1
Grupos rechazados: 1
Personas admitidas: 650
Capacidad máxima: 700
Espacios disponibles: 50
```

>>>

Semana 07

¿Qué vimos?	¿Qué aprendí?	¿Cómo lo puedo aplicar?
Esta semana vimos un poco mas de Python, trabajamos con listas, diccionarios, tuplas y conjuntos. También usamos Visual Studio Code para correr los programas y revisamos un ejemplo de varias sedes con ventas y metas.	Aprendí que las listas se pueden cambiar, los diccionarios guardan datos por nombre, las tuplas no se modifican y los conjuntos no repiten información. Al principio me confundí un poco con algunas cosas, pero haciendo los ejercicios ya fui entendiendo mejor.	Esto me puede servir para ordenar datos de ventas, revisar metas y sacar información sin hacerlo todo a mano. En el trabajo me serviria mucho para tener los datos mas organizados y evitar repetir cosas.

Evidencias

Esta semana hicimos varios ejercicios cortos para practicar las estructuras de datos. También trabajamos con un ejemplo de emprendimientos de Costa Rica y sus ventas. A continuación dejo los trabajos que hicimos y los archivos de la actividad.

Detalle de las actividades

Verificar entorno: Probamos Python en Visual Studio Code para ver que corriera bien y revisar la versión y la ruta del interprete.

Microreto 1: lista de ventas: Cambiamos la venta del segundo día y vimos como cambia el total de la lista.

Microreto 2: diccionario de meta: Revisamos los datos de una sede y practicamos cambiar el valor de una meta usando su clave.

Microreto 3: tupla de resultado: Vimos que una tupla no se cambia directamente, entonces hay que crear otra con el dato corregido.

Microreto 4: conjunto de provincias: Agregamos provincias a un set y comprobamos que una provincia repetida no aumenta la cantidad.

Sedes de EmprendeCR: Usamos datos de diez sedes ficticias, con provincias, tipos de negocio, ventas y metas.

Análisis de sedes: Importamos la lista de emprendimientos y revisamos cuantas sedes hay y el nombre de la primera.

En las siguientes páginas van las evidencias originales de cada actividad.

Notebook PDF export

Kernel: Python 3 • Code cells: 1 • Markdown cells: 1

Primera ejecución en Visual Studio Code: comprobar Python y la terminal.

In [2]:

```
# sys solo permite mostrar qué Python ejecutó este archivo.
# Su uso se explicará con detalle cuando se estudien los módulos.
import sys

print("Python ejecutó este archivo correctamente")
print("Versión de Python:", sys.version.split()[0])
print("Ruta del intérprete:", sys.executable)

# Cambie el mensaje del primer print(), guarde y ejecute otra vez.
# La versión y la ruta deben corresponder al intérprete seleccionado en VS Code.
```

Generated by Jupy Tools · 10/28/2026

Notebook PDF export

Kernel: Python 3 • Code cells: 1 • Markdown cells: 1

Microreto 1: recuperar lo aprendido sobre listas.

Prediga la salida, modifique una posición y explique el cambio.

In [2]:

```
# Una lista conserva el orden: índice 0 = primer día, índice 1 = segundo día.
ventas = [85000, 92000, 78000]
print("Venta del segundo día:", ventas[1])
print("Total inicial:", sum(ventas))

# La venta del segundo día se corrigió de 92000 a 100000.
# TODO: actualice la posición adecuada con una asignación por índice.
ventas [1]= 100000
print("Segundo día corregido:", ventas[1])
print("Total corregido:", sum(ventas))


# Para explicar: ¿por qué no cambió la cantidad de elementos?
# Para comprobar: el total pasa de 255000 a 263000; aumenta 8000.
```

Generated by Jupy Tools · 10/28/2026

Notebook PDF export

Kernel: Python 3 • Code cells: 1 • Markdown cells: 1

Microreto 2: leer y cambiar un campo de un diccionario.

Un diccionario guarda campos con nombre; cada nombre es una clave.

In [2]:

```
# A diferencia de ventas[1], aquí se usa una clave como sede["meta"].
# La lista de ventas puede ser uno de los valores del diccionario.
sede = {
    "nombre": "Soda San Pedro",
    "provincia": "San Jose",
    "ventas": [85000, 92000, 78000],
    "meta": 450000,
}
print("Nombre:", sede["nombre"])
print("Primera venta:", sede["ventas"][0])
print("Meta inicial:", sede["meta"])

# La meta registrada cambió a 470000.
# TODO: actualice solo el valor asociado con la clave "meta".
sede ["meta"] = 4700000
print("Meta corregida:", sede["meta"])
print("Nombre después del cambio:", sede["nombre"])
print("Total de ventas", sum(sede["ventas"]))
# Para explicar: ¿por qué el nombre y las ventas siguen iguales?
# Para comprobar: la meta pasa de 450000 a 470000.
```

Notebook PDF export

Kernel: Python 3 • Code cells: 1 • Markdown cells: 1

Microreto 3: crear otro par cuando cambia un resultado.

Una tupla conserva el orden como una lista, pero sus posiciones no se cambian.

In [2]:

```
# Este par guarda nombre y total semanal; 0 es el nombre y 1 es el total.
par_original = ("Soda San Pedro", 462000)
print("Nombre:", par_original[0])
print("Total original:", par_original[1])

# Otro ejemplo de la misma forma sería ("Panaderia Cartago", 351000).
# Se corrigió el total de Soda San Pedro a 470000.
# TODO: reemplace la línea provisional por una TUPLA NUEVA con nombre y total.
# No use par_original[1] = 470000: eso intenta modificar una tupla.
par_corregido = par_original
print("Total original después:", par_original[1])
print("Total del nuevo par:", par_corregido[1])

# Para explicar: ¿por qué se necesita un par nuevo?
# Para comprobar: el original conserva 462000; el nuevo muestra 470000.
```

Generated by Jupy Tools · 10/28/2026

Notebook PDF export

Kernel: Python 3 • Code cells: 1 • Markdown cells: 1

Microreto 4: guardar provincias sin repetirlas.

Un set conserva valores únicos. Se agrega con add(), sin posición ni índice.

In [2]:

```
# También se puede iniciar vacío con set(); {} vacío crea un diccionario.
provincias = {"San Jose", "Cartago"}
print("Cantidad inicial:", len(provincias))
print("¿Está San Jose?:", "San Jose" in provincias)

# TODO: agregue otra vez "San Jose" mediante add().
provincias. add ("San Jose")
print("Después de repetir San Jose:", len(provincias))


# TODO: agregue "Heredia" mediante add().
provincias. add ("Heredia")
print("Después de agregar Heredia:", len(provincias))
print("¿Está Heredia?:", "Heredia" in provincias)

# Para explicar: ¿por qué repetir San Jose no aumenta la cantidad?
# Para comprobar: las cantidades son 2, 2 y 3; Heredia sí es un dato nuevo.
# Si imprime el set, el orden de las provincias puede variar.
```

Notebook PDF export

Kernel: Python 3 • Code cells: 1 • Markdown cells: 1

Diez sedes ficticias EmprendeCR para analizar en un archivo nuevo.

Las primeras seis proceden del caso anterior; las últimas cuatro amplían el escenario de práctica. Mantenga los registros sin cambios durante la actividad.

In [2]:

```
emprendimientos = [
    {
        "nombre": "Soda San Pedro",
        "provincia": "San Jose",
        "tipo": "Alimentacion",
        "ventas": [85000, 92000, 78000, 110000, 97000],
        "meta": 450000,
    },
    {
        "nombre": "Panaderia Cartago",
        "provincia": "Cartago",
        "tipo": "Panaderia",
        "ventas": [65000, 70000, 72000, 69000, 75000],
        "meta": 380000,
    },
    {
        "nombre": "Cafeteria Liberia",
        "provincia": "Guanacaste",
        "tipo": "Cafeteria",
        "ventas": [98000, 105000, 99000, 120000, 115000],
        "meta": 500000,
    },
    {
        "nombre": "Feria de Heredia",
        "provincia": "Heredia",
        "tipo": "Feria",
        "ventas": [72000, 76000, 80000, 78000, 74000],
        "meta": 390000,
    },
    {
        "nombre": "Marisqueria Puntarenas",
        "provincia": "Puntarenas",
```

```
        "tipo": "Restaurante",
        "ventas": [115000, 108000, 112000, 118000, 121000],
        "meta": 560000,
    },
    {
        "nombre": "Tienda Escazu Local",
        "provincia": "San Jose",
        "tipo": "Comercio",
        "ventas": [88000, 91000, 87000, 94000, 90000],
        "meta": 460000,
    },
    {
        "nombre": "Libreria Alajuela",
        "provincia": "Alajuela",
        "tipo": "Comercio",
        "ventas": [90000, 95000, 100000, 85000, 105000],
        "meta": 480000,
    },
    {
        "nombre": "Tienda Limon Centro",
        "provincia": "Limon",
        "tipo": "Comercio",
        "ventas": [70000, 80000, 75000, 85000, 90000],
        "meta": 390000,
    },
    {
        "nombre": "Ferreteria Desamparados",
        "provincia": "San Jose",
        "tipo": "Ferreteria",
        "ventas": [50000, 60000, 55000, 65000, 70000],
        "meta": 400000,
    },
    {
        "nombre": "Vivero Paraiso",
        "provincia": "Cartago",
        "tipo": "Vivero",
        "ventas": [65000, 70000, 60000, 75000, 80000],
        "meta": 350000,
    },
]
```

Notebook PDF export

Kernel: Python 3 • Code cells: 1 • Markdown cells: 0

In [1]:

```
from sedes import emprendimientos
print ("Sedes:", len(emprendimientos))
print("Primer emprendimientos: ", emprendimientos[0]["nombre"])
```

Generated by Jupy Tools · 10/28/2026